

Imagine that you have a large pile of pebbles. You are separately concerned with your recent acquisition of twenty cattle, and are looking to divide them into herds of different and unique sizes to distribute among your five children in a way that obviously signifies favoritism. Given that, you still want to have a relatively level playing field, and each child should only have one more cow than the next favorite child. There is one final issue: you must give the cattle out publicly, in view of all children.

This means that once you have started giving one of your children cattle and give cattle to another child, the first child will storm off, knowing that they are not your favorite, and will be unable to receive more cattle. In short, you must give a child all of their cattle at once, and so you need to know how many they will receive beforehand. This problem has haunted you for a couple of weeks, and your children are getting restless. Talk is brewing of an even split of cattle, something that you cannot abide. How can you know what size each herd should be?

Then, it strikes you: *Aha* – your pile of pebbles. If you pretend that twenty of your pebbles are cows, you can arrange *them* into “herds,” and give them out to imaginary children. Once you settle on an arrangement, you can bring each child’s pile of pebbles with you, depleting them one by one as you dole out cattle. Solution in mind, you begin to distribute pebbles in ascending count between your five piles.

The next day, you distribute the cattle according to your pebble system, and everything goes to plan. Your favorites are obviously favored, but your least are not left in

the dust. You have just performed a computation, one that has secured the peaceful future of your family.

Computation is, at its core, the transformation of information. Computation works by assigning symbolic value to objects that operate in a structured manner similar to the real phenomena one is trying to predict, manipulating those objects according to a set of rules, and observing the outcome. Computation is an ancient human activity, because it allows us to cheaply make decisions and operate in the world around us.

Modern computation works in the same way, but it has access to an unprecedented amount of fast, easily manipulable and stable objects. These objects are called bits, and they can exist in two states, either zero or one. While bits are less obviously symbolic of anything at all, very smart people like Claude Shannon and Alan Turing realized that, given enough bits, you could construct arbitrarily complicated logical machines to solve any solvable problem.

If you wanted to split your cows with the aid of a computer, you could assign a group of bits to each of your children. Then, you could add 1 to the first, 2 to the second, and so on, keeping track of the total added, no rocks required. This is trivial for a modern computer, and if you so desired, it would not be difficult to simulate a scenario where you distribute millions of cows among thousands of children, and in fact my computer did so in well under a second.

Computers excel at these types of problem. Their ability to represent incredibly complex systems with symbols of negligible size and cost, and speed approaching the

speed of light, has brought us into the information era. However, there are problems that take time even for computers to solve. Among these are search problems, problems that involve looking through a large list of possible solutions and finding the right one.

Your children have just stumbled onto one such problem. They must arrange their cows, each assigned a unique power of two, so that when multiplied by their position in the lineup and summed, the total matches a given number. Only then will they win glory and wealth.

You have recently invested in a powerful computer, understanding the utility of such a device, and feel up to the challenge. This problem seems large, but your computer should be more than powerful enough to deal with it. Twelve hours later, and you have checked barely a trillionth of a percent of the possible solutions. At this rate, you and everyone you know will be long gone by the time an answer is reached.

With only classical computers, large search problems are intractable. Due to the nature of the problem, the computer must check every single answer until it finds the correct one. On average, this means looking through half of all possible solutions. Computers can complete billions of operations per second, but if your search space is $20!$, or roughly 2.4×10^{18} , you will still have to wait thousands of years for an answer on average.

This is where quantum computing comes in. Quantum computers take advantage of quantum effects to solve problems in a way that classical computers fundamentally cannot, opening up entirely new avenues of exploration. To understand quantum

computing, understanding of two major quantum phenomena is essential: superposition and entanglement.

Superposition is a property shared by all quantum particles that allows them to functionally exist in multiple states at once, until interacted with. Particles in superposition have probabilities to be in different states, and when interacted with, will probabilistically “choose” a state. This makes no intuitive sense, and for the purposes of this explanation, it will continue to do so.

Entanglement is a property that allows quantum particles to inextricably link their states. When one entangled particle is interacted with, collapsing its state, the other will as well. Additionally, entangled particles exist in superpositions of all possible combinations of their states.

Just like bits or anything else can be symbolic for the purpose of computing, we can assign values to quantum particles. Such particles are called qudits, which are quantum particles that can exist in a set number of states that we define. Most common among these are qubits, which can exist in two states.

Single qubits not in superposition are represented by column vectors, with 0 and 1 both corresponding to their own vectors, as figure 1. These are also called basis vectors. When qubits are combined, they can be represented by larger column vectors, with the n th entry being 1 for any n , counting from 0. For example, figure 2 shows the number 2 using 2 qubits

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Fig. 1: The 0 and 1 basis vectors for 1 qubit

$$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Fig. 2: The 2 basis vector for 2 qubits

What makes qubits interesting is that they can exist in a superposition of states. For example, a very simple entangled qubit is shown in figure 3. This qubit is halfway in the zero state and halfway in the one state, although it is not immediately obvious why. Where did $\sqrt{2}$ come from?

When a qubit is interacted with and observed, all entries are squared, and their value is equal to the probability that the qubit will be observed as the basis vector with 1 at that position. In this case, the square of both entries is 0.5, and so this qubit has an equal chance to be observed in the 0 state and the 1 state. Qubits can be biased towards certain observations, and figure 4 shows a qubit that is quite likely to be observed in the 1 state.

$$\begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$$

Fig. 3: a qubit in superposition.

$$\begin{bmatrix} \frac{1}{100} \\ \frac{\sqrt{9999}}{100} \end{bmatrix}$$

Fig. 4: a qubit in a 1-leaning superposition

Qubits are operated on by logical gates that can be represented by complex-valued matrices. These matrices must be unitary, which means that they must be invertible, and their inverse must be equal to their conjugate transpose, both of which will be explained. These functions must be unitary because information at the quantum level cannot be destroyed, and a non-unitary function necessarily does not preserve all given information, since it cannot be reversed.

For a matrix to be invertible, it must be square, so it has an equal number of rows and columns, and there must exist some matrix which reverses any matrix multiplication that it performs. For many matrices, this is the matrix itself. An example of this is the Pauli-X gate, which is shown in figure 5. When multiplying the 0 basis vector by this gate, we get the 1 basis vector, as shown in figure 6. When we apply the inverse of the gate, which happens to be itself, to the result again, as shown in figure 7, we get back to 0. Thus, the Pauli-X gate is invertible, because there exists a matrix that reverses it.

An example of a non-invertible function is shown in figure 8. This matrix always sets the second row in our vector to 0, no matter the input. That means that multiple inputs are transformed into the same output, and information is lost.

Our matrices must not only be invertible, but their conjugate transpose must be equal to their inverse. A conjugate transpose is found by first finding the transpose of a matrix, or flipping it along the top-left to bottom-right diagonal, and then flipping the signs of any complex components of the entries. If this new matrix is equal to the inverse of the original matrix, congratulations! You have just found a valid quantum operator.

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Fig. 5 The Pauli-X gate

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} (0)(1) + (1)(0) \\ (1)(1) + (0)(0) \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Fig. 6 Application of the Pauli-X gate to the 1 basis vector

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} (0)(0) + (1)(1) \\ (1)(0) + (0)(1) \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Fig. 7: Application of the Pauli-X gate to the 0 basis vector

$$\begin{bmatrix} 1 & -i \\ 0 & 0 \end{bmatrix}$$

Fig. 8: A non-invertible matrix

There are infinitely many possible quantum operators, but there are a few especially interesting ones that see regular use in proposed algorithms. One of these is the

aforementioned Pauli-X gate, which flips the qubit from a 0 to a 1, and vice versa. Another is the Pauli-Y gate, shown in figure 9, which acts similarly to the Pauli-X gate but introduces a positive or negative complex component. Figure 10 shows the Pauli-Z gate, which leaves 0 unchanged, but flips the sign of 1.

$$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

Fig. 9: Pauli-Y gate

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Fig. 10: Pauli-Z gate

These operators are all valid, but don't do much on their own. The Hadamard operator, shown in figure 11, opens up the interesting parts of quantum computing. The Hadamard operator takes a given qubit and puts it into a 50/50 superposition.

$$\begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}$$

Fig. 11: the Hadamard operator

Now that you have a solid understanding of the fundamentals of quantum computing, you need a way to help your children determine what the correct lineup of cows is. Your search space is $20!$, and so you will need at least $\log_2(20!)$ qubits, which comes out to 62 qubits. Next, you will need to create a massive superposition of these qubits, each possible state representing one possible permutation. Then, you can create an “oracle”

function, which looks at each permutation, finding the one that satisfies the given number. After that, you can observe the entanglement, and get the answer.

Wait – aren't superpositions probabilistic? There is a near zero percent chance that you get the right number. Even if your circuit knows the right answer, it cannot directly communicate that information to you. To extract useful information, you'll have to make it so that the superposition always yields useful results. What if you made it nearly 100% likely that you were given the right answer by applying a Hadamard-like gate that had an unbalanced set of probabilities? This is a good idea, but it runs into a few issues. First, it is probably not a unitary operator. Second, even if it is, the computer must know which operator to use based on the state, which it cannot without knowledge it can't have.

That won't work, but what if you added an extra step to your algorithm that changed all qubits into the correct answer, so no matter what you observe, it is useful? Sadly, this operation is very un-unitary. There is no way to turn many inputs into the same output in a way that doesn't destroy information, and so this will not work. You throw your hands up in exasperation, but as you are about to abandon your children to lose the competition, Lov Grover comes down from on high, solution in hand.

For unstructured search problems, there is an algorithm that will take on average $\sqrt{n}$ iterations to find an answer, or a quadratic speedup. This is Grover's algorithm, and it works using an ingenious method of iterated amplifications that bring the superposition very close to 100% likely to return the correct answer when observed. Grover's algorithm shares the same basic intuition with your solution.

First, it creates a massive superposition. Then, it applies an oracle function, but in a very unintuitive way. All that the oracle function does is flip the sign of the correct qubit. Because we square all values in our qubits during observation, negative values don't matter for observation. Why does the sign flip matter? The next step, called the diffusion operator, is where Grover's algorithm comes together. Grover computes the average amplitude of each qubit, which is equal to $\frac{(n-2)(\frac{1}{\sqrt{n}})}{n}$, since initial amplitudes are $\frac{1}{\sqrt{n}}$, and one amplitude is negative, meaning that we sum up $n-2$ amplitudes and divide by n . Then, he takes twice the average, and for each amplitude, subtracts amplitude from twice the average. For all positive amplitudes, this brings the amplitude closer to zero. For the negative amplitude, since subtracting a negative adds, it becomes closer to one.

All of this is done while maintaining reversibility, and the operator is unitary. If we repeat this operation $\sqrt{n}$, or roughly 1.56 billion times, we reach a probability very close to 1. Assuming roughly similar execution speed for our classical and quantum computers, this completes roughly one and a half billion times faster. Assuming an ideal quantum computer, this would only take around a fifth of a second! Your children are saved!

Although, what was that about an ideal quantum computer?

Quantum computers are extremely difficult to realize in the real world. Keeping a superposition for even a tenth of a second is difficult, and that's without the extremely precise interactions that threaten to break our superposition. Additionally, we have been talking about quantum operator matrices as if they exist in some infinite set, but at small enough a scale everything is discrete, and error exists. While quantum theory allows for an

infinitesimally slight shift of a qubit, real life instruments are only so precise, and we can't use information past that level of precision.

This is a bit like looking at a classical computer, saying that $V = IR$, that all three of those values are real-valued, and so a piece of copper wire is an infinitely precise computational machine that can perform arbitrary multiplication. The single largest issue with quantum computers is that they exist in real life, which severely limits their capabilities.

Once we reconcile this, though, quantum computers will change the world.

IBM. (n.d.). *What is quantum computing?* IBM. Retrieved June 10, 2025, from <https://www.ibm.com/think/topics/quantum-computing>

TechTarget. (n.d.). *Classical computing*. WhatIs.com. Retrieved June 10, 2025, from <https://www.techtarget.com/whatis/definition/classical-computing>

Britannica. (n.d.). *Mathematics in ancient Egypt*. In *Encyclopaedia Britannica*. Retrieved June 10, 2025, from <https://www.britannica.com/science/mathematics/Mathematics-in-ancient-Egypt>

Nielsen, M. A., & Chuang, I. L. (2002). *Quantum Computation and Quantum Information* (Thesis, MIT). Massachusetts Institute of Technology. <https://dspace.mit.edu/handle/1721.1/11173>

GeeksforGeeks. (n.d.). *Introduction to Grover's algorithm*. Retrieved June 10, 2025, from <https://www.geeksforgeeks.org/introduction-to-grovers-algorithm/>

Wolchover, N. (2023, June 20). *How does the quantum world cross over?* *Scientific American*. <https://www.scientificamerican.com/article/how-does-the-quantum-world-cross-over/>

D'Ariano, G. M., Perinotti, P., & Tosini, A. (2020). Emergence of space-time from topologically homogeneous causal networks. *Frontiers in Physics*, 8, 589504. <https://www.frontiersin.org/articles/10.3389/fphy.2020.589504/full>

Quantiki. (n.d.). *Quantum gates*. Retrieved June 10, 2025, from <https://www.quantiki.org/wiki/quantum-gates>

Indian Institute of Technology Delhi. (n.d.). *Distinguished alumnus award – Professor Arvind (2022)*. Retrieved June 10, 2025, from <https://alumni.iitd.ac.in/distinguished-alum-award/394>

McKie, R. (2024, May 30). *Quantum Schrödinger's cat survives for a stunning 23 minutes*. *New Scientist*. <https://www.newscientist.com/article/2453356-quantum-schrodingers-cat-survives-for-a-stunning-23-minutes/>